

Tool Use

How a model calls a function · video 2:17

This lesson answers one question: how can a model that only reads and writes text check the weather, search the web or run code? It never runs anything itself. The app **describes the tools** in the context, the model **writes a request** in a trained format, the app **runs the function**, and the **result goes back into the context** for the model to continue. You control which tools exist, so you control what the model can do.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 7).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. The model only writes requests	▶ 0:08-0:28	2	☐
2. The four steps	▶ 0:28-1:25	3	☐
3. When the model does not call	▶ 1:25-1:43	2	☐
4. The loop, and who is in control	▶ 1:43-2:14	2	☐

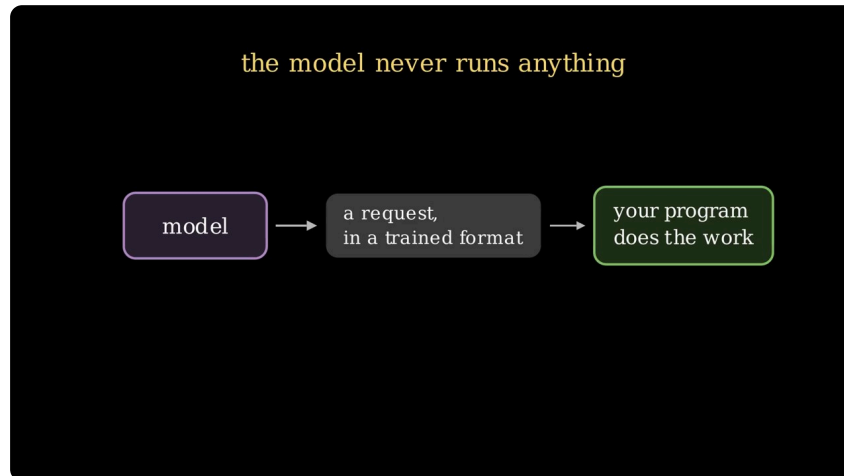
Before you start: episode 1 of this series (chat templates and context). Code: `code/p07_tool_use` (downloads Qwen2.5-1.5B-Instruct, about 3 GB; runs on a CPU).

CONCEPT 1 OF 4

The model only writes requests

▶ Video 0:08–0:28

Ask a model about today's weather and it cannot know: no window, no internet, no clock, just **text in and text out**. Yet assistants check the weather, search the web and run code. The trick: the model **never runs anything itself**. It writes a **request**, in a format it was trained on, and **your program** does the work.



▶ 0:26 The model writes a request in a trained format; your program does the work.

KEY IDEA

Tool use = the model writes structured text asking for an action; ordinary code decides whether and how to perform it.

Check 1: The model only writes requests

Answer all 2 questions before looking at the answers. Pass mark: all correct.

1.1 TRUE OR FALSE

“When a model calls a weather tool, the model itself connects to the weather service.”

True ☐ False ☐

1.2 MULTIPLE CHOICE

Where does the model learn the format for writing tool calls?

- A. It invents a new format each time
- B. From training (fine-tuning on examples) plus the instructions in the system prompt
- C. From the weather service
- D. From the user

CONCEPT 2 OF 4

The four steps

▶ Video 0:28-1:25

1. Describe the tools. A name, what it does and its parameters, written as JSON. The chat template pastes this into the **system prompt** with instructions: you may call functions, here are their signatures, here is how to write a call. It is just more text in the context.

1 · describe the tools

```
def get_weather(city: str):  
    """Get the current weather in a city."""
```

in the system prompt (real Qwen2.5 template)

```
# Tools  
You may call one or more functions to assist with the user query.  
You are provided with function signatures within <tools></tools> XML tags:  
<tools>  
{  
  "type": "function",  
  "function": {  
    "name": "get_weather",  
    "description": "Get the current weather in a city.",  
    "parameters": {  
      "type": "object",  
      "properties": {  
        "city": {  
          "type": "string",  
          "description": "The name of the city, e.g. Milan",  
          "required": true  
        }  
      }  
    }  
  }  
</tools>  
For each function call, return a json object with function name and arguments within <tool_call></tool_call> XML tags: _
```

just more text in the context

▶ 0:44 Step 1: the real Qwen2.5 template pastes the tool's JSON description into the system prompt.

2 · the model writes a call

What is the weather like in Milan right now? Do I need an umbrella?

```
<tool_call>  
{  
  "name": "get_weather",  
  "arguments": {  
    "city": "Milan"  
  }  
</tool_call>
```

the model's real reply

▶ 0:55 Step 2: instead of an answer, the model writes a tool call.

2. The model writes a call. Asked “What is the weather like in Milan right now? Do I need an umbrella?”, the model's reply is not an answer but `<tool_call> {"name": "get_weather", "arguments": {"city": "Milan"}} </tool_call>`. **3. The app runs it:** it spots the tool call, reads the JSON and runs the real function (here a made-up service: 18 °C, light rain). This is the only moment anything happens in the world. **4. The result goes back** into the context as a tool response; the model now reads **264 tokens**, including the weather, and answers.

3 · your program runs it

```
graph LR  
    A[parse JSON] --> B[get_weather("Milan")]  
    B --> C[{"city": "Milan", "temperature_c": 18, "sky": "light rain"}]
```

a made-up weather service

the only moment anything happens in the world

▶ 1:09 Step 3: the program parses the JSON and runs the real function.

4 · the result goes back into the context

```
system: tools _  
user: What is the weather like in Milan right _  
assistant: <tool_call> get_weather(Milan) </tool_call>  
tool: <tool_response> 18 °C, light rain </tool_response>  
assistant: _
```

264 tokens

The current temperature in Milan is 18°C and it's currently experiencing light rain. It might be raining lightly so you might want to bring an umbrella if you plan on going out.

▶ 1:24 Step 4: the result goes back into the context, and the model answers.

KEY IDEA

Describe → call → run → return. Everything except “run” is text in the context.

Check 2: The four steps

Answer all 3 questions before looking at the answers. Pass mark: all correct.

2.1 PUT IN ORDER

Put in order: *the app runs the function · the model writes a tool call · the tool descriptions are added to the prompt · the model answers · the result is appended to the context.*

.....

2.2 CALCULATION

The first request was 204 tokens; after the tool call and its result were added, 264. How many tokens did the call and the response add?

.....

2.3 SHORT ANSWER

Why does the function's docstring matter?

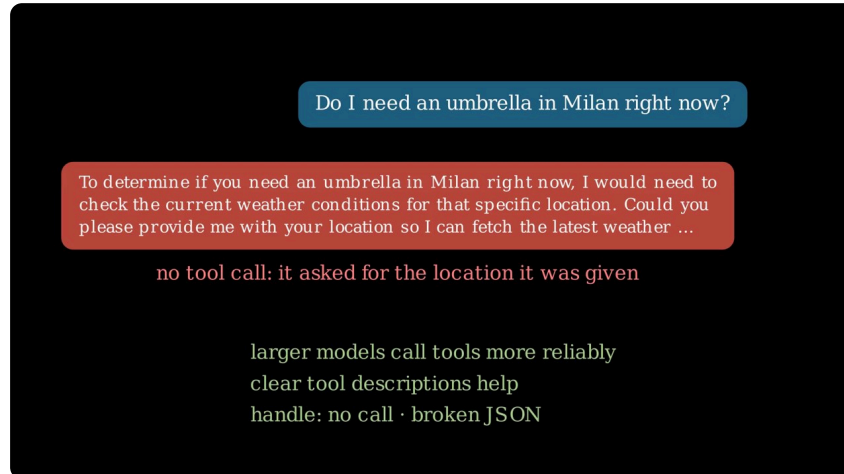
.....

CONCEPT 3 OF 4

When the model does not call

▶ Video 1:25-1:43

Ask more vaguely, “*Do I need an umbrella in Milan right now?*”, and the same small model did not call the tool at all: it asked for your location, even though you had said Milan. Larger models call tools more reliably, and clear descriptions help. Your code must handle **no call**, and **broken JSON**.



▶ 1:42 A vaguer question: the 1.5B model does not call the tool and asks for the location it was given.

KEY IDEA

Whether to call a tool is the model's guess. Robust apps expect missing, wrong or malformed calls.

Check 3: When the model does not call

Answer all 2 questions before looking at the answers. Pass mark: all correct.

3.1 MULTIPLE CHOICE

The model's reply contains `<tool_call>` but the JSON inside is invalid. What should the app do?

- A. Crash
- B. Run the function with empty arguments
- C. Not run anything; report the error or ask the model to try again
- D. Guess the arguments

3.2 SHORT ANSWER

Give one change to the tool description that might make the model call `get_weather` for umbrella questions.

.....

CONCEPT 4 OF 4

The loop, and who is in control

▶ Video 1:43-2:14

In code it is a **loop**: generate; if there is no tool call, you are done; otherwise parse it, run the function, append the result, and generate again.

```
1 while True:
2     reply = generate(messages)
3     if "<tool_call>" not in reply:
4         return reply
5     call = json.loads(between(reply, "<tool_call>", "</tool_call>"))
6     result = tools[call["name"]](**call["arguments"])
7     messages += [tool_call_message(call), tool_response(result)]
```

▶ 1:51 Tool use is a loop: generate, run any call, append the result, generate again.

the model chooses what to call · you choose what it can do

get_weather
read-only: harmless

send_email · make_payment
actions: real consequences

- ✓ ask the user to confirm
- ✓ check the arguments
- ✓ never run model code without limits

▶ 2:06 Read-only tools are harmless; actions need confirmation, checked arguments and limits.

The model chooses what to call, but **you choose what it can do**. Reading the weather is harmless. Sending an email or making a payment is not: **ask the user to confirm, check the arguments**, and never run code from the model without limits. Give a model several tools and a goal, run this loop until the job is done, and you have an **agent** (next episode).

KEY IDEA

The tool list is the model's set of powers. Grant only what is needed, and guard every action with consequences.

Check 4: The loop, and who is in control

Answer all 2 questions before looking at the answers. Pass mark: all correct.

4.1 MULTIPLE CHOICE

Which tool most needs a confirmation step before it runs?

- A. get_weather(city)
- B. search_docs(query)
- C. transfer_money(to, amount)
- D. get_time()

4.2 HANDS-ON CODE

Try it yourself. Replace the two `chat(...)` calls at the end of `tool_use.py` with the lines below and run it. (a) Which question triggers a tool call, with what arguments? (b) What does the model do with the other one, and why is that correct?

```
chat("Is it raining in Paris right now?")
chat("Write a haiku about rain.")
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. The model only writes requests · rewatch ▶ 0:08-0:28

1.1 False. The model only writes the request; the app's code calls the service.

1.2 B. Tool-capable models are fine-tuned on tool-call examples, and the template restates the format.

2. The four steps · rewatch ▶ 0:28-1:25

2.1 tool descriptions added → model writes a tool call → app runs the function → result appended → model answers. These are the four steps, plus the final answer.

2.2 60. $264 - 204 = 60$, including the template's tags and markers.

2.3 It becomes the tool's description in the prompt, which is how the model decides when and how to call it. The template turns the signature and docstring into the JSON the model reads.

3. When the model does not call · rewatch ▶ 1:25-1:43

3.1 C. Never act on a request you could not parse.

3.2 For example: “Get the current weather in a city, including rain. Use it for questions about rain, umbrellas or what to wear.” The model decides from the description; mentioning the use cases makes the match easier.

4. The loop, and who is in control · rewatch ▶ 1:43-2:14

4.1 C. It changes the world irreversibly; the others only read information.

4.2 (a) “Is it raining in Paris right now?” → get_weather with {"city": "Paris"}; the fake service says light rain, and the model answers “Yes, it is currently raining in Paris...”. (b) For the haiku it answers directly, with no tool call: writing a poem needs no outside information. The model decides per question whether a tool is useful. (The fake service returns the same weather for every city.)